Ingeniería de Software Asistida por IA

Tema 5: Ciberseguridad, FinOps y Criterio de Recursos

Guía de Estudio Detallada para Estudiantes

Paranoia Absoluta: El Paradigma de la Seguridad con IA

El objetivo principal de un LLM es generar código que resuelva tu requerimiento visualmente. El camino algorítmico más rápido para lograr que algo "funcione" implica, casi invariablemente, **desactivar o ignorar las capas de seguridad**.

Si le pides a una IA que implemente un sistema de usuarios, configurará permisos globales abiertos para evitar los errores de *CORS* o de autenticación que le impedirían completar la tarea rápido. Como Arquitecto, tu estado mental al auditar código generado por IA debe ser el de un atacante: asume siempre que la máquina dejó las puertas traseras abiertas por conveniencia matemática.

El Ataque DevTools y la Fuga de Datos

En tecnologías modernas como Firebase o Supabase, la base de datos se puede consultar directamente desde el Frontend (React). Esto es ágil, pero aterrador si no se audita.

El Ataque: Si la IA genera un buen código visual de lista de usuarios, pero "olvida" asegurar la tabla, un atacante puede abrir su navegador en Modo Incógnito, presionar F12 (DevTools), acceder a la consola de JavaScript y ejecutar: `await supabase.from('users').select('*')` Sin hackear contraseñas, el atacante acaba de descargar tu base de datos completa.

La Solución: Obligar a la IA a proveer siempre los scripts SQL para activar las **Row Level Security (RLS)**, garantizando que el usuario solo pueda leer las filas que le pertenecen criptográficamente mediante su JWT.

API Keys: El "God Mode" Expuesto

Plataformas como Supabase entregan dos llaves: la anon_key (pública, segura de usar en React si el RLS está activado) y la `service_role_key` (la llave "Modo Dios" que ignora toda la seguridad).

Frecuentemente, para sortear un problema temporal de permisos, la IA te sugerirá colocar la `service_role_key` en tu código Frontend. **Si haces esto, tu infraestructura está comprometida**. Cualquier visitante de la web puede extraer esa llave de tu código compilado y borrar tus bases de datos. **Regla:** Las llaves secretas o administrativas jamás deben tocar el Frontend. Deben vivir exclusivamente como Variables de Entorno (`.env`) en el backend.

Arquitectura S2S (Server-to-Server)

Cuando tu aplicación necesita hacer operaciones críticas, como procesar pagos con tarjeta de crédito (ej. Stripe, Transbank), enviar correos masivos o consumir APIs de pago (como la de OpenAI), **el Frontend jamás debe hacer la petición (`fetch`) directamente**. Si lo hace, la llave de pago viajará al navegador del cliente y podrá ser robada en la pestaña "Network" de DevTools.

El Escudo S2S: Debes instruir a la IA a crear un puente (Proxy). 1. React (Frontend) hace una petición simple a *Tu Backend* (Edge Function). 2. Tu Backend (en un entorno sellado) usa la llave secreta para comunicarse con la API de Stripe. 3. El Backend devuelve solo un mensaje de éxito a React.

FinOps y Rate Limiting (Protegiendo la Billetera)

En la nube, un ataque no siempre busca robar datos; a veces busca quebrar a la empresa financieramente. Si aplicaste la arquitectura S2S perfectamente, tu llave de pago está segura. Pero, ¿qué pasa si un bot ataca tu Edge Function haciendo 10,000 peticiones por segundo? La función ejecutará 10,000 llamadas a la API de OpenAI usando tu propia infraestructura legal, y tú pagarás esa factura masiva.

Defensa de Infraestructura: Debes exigirle a la IA que implemente **Rate Limiting** (límite de peticiones por IP).

> *"En esta Edge Function, antes de llamar al servicio externo de costo, implementa un mecanismo de Rate Limiting para rechazar operaciones si una misma IP hace más de 5 peticiones por minuto."*

Criterio de Recursos y Desperdicio

El último pilar del Arquitecto es entender dónde debe ocurrir el procesamiento. La IA tenderá a pedir grandes volúmenes de datos a la base de datos para filtrarlos o sumarlos mediante ciclos `for` o `reduce` en el Frontend (React).

Esto desperdicia ancho de banda, revienta la RAM del celular del usuario y genera costos masivos de lectura en la base de datos. **El Criterio:** Las bases de datos están hiper-optimizadas para las matemáticas. Exige a la IA que use funciones nativas de SQL (como `SUM()`, `GROUP BY`) o RPCs (Remote Procedure Calls) en Supabase para que el cálculo se haga en el servidor, devolviendo al Frontend únicamente el número final resultante.